# CSC108H5 – Fall 2025: Midterm

University of Toronto Mississauga

**Date:** October 23rd, 2025 **Time:** 7:10 - 9:00 p.m.

**Instructors:** Profs. M. Liut, M. H. Vu, J. Jung, M. Mahmoud

**Head TAs:** N. Sibia, R. Ye

---

Do **NOT** turn this page until you have received the signal to start.
Please fill out your information and read the instructions below. Good luck!

---

**UTORid is alphanumeric**, e.g., student Jane Doe's UTORid is `doejan159`.

---

The University of Toronto Mississauga and you, as a student, share a commitment to academic integrity. You are reminded that you may be charged with an academic offence for possessing any unauthorized aids during the writing of a midterm. **This is a closed book midterm**. Make sure the only items at your station are your TCard/ID, writing implements, and a bottle of water.

To maintain fairness, **we will NOT answer any questions**. You are not permitted to communicate in any way about the questions or answers, except with a Teaching Assistant or Instructor. Any unauthorized communication will be considered a case of academic misconduct.

If you use any space for rough work, indicate clearly what you want marked. Multiple choice answers must be placed on the bubble sheet. For an extra point on this midterm, draw your a pineapple on the top right corner of page four. Do not look around or ask for confirmation. Doing so will cause you to forfeit the point. You must follow all question specific requirements and are only permitted to use material taught in lecture up to Week 8 (excluding dictionaries and tuples).

This midterm is composed of **34 questions** (22 multiple-choice, 5 fill-in-the-blanks, 3 Parsons problems, 3 (technically 4) written answers, and 1 bonus) on 28 total pages. We are using Python 3.11 as the programming language for this midterm and the aid sheet is attached. Please make sure your midterm contains all of the questions prior to starting.

## Part 1: Multiple Choice Questions (MCQ).

Do **NOT** put your final answers here. You **MUST** fill in your answers on the bubble sheet at the back of this test.

Below you will find Multiple-Choice Questions (MCQ). **For each question you must select ONE answer and fill it in on the bubble sheet at the back of this exam**. Failure to do so will result in a mark of 0. We do not employ a negative marking scheme, so do not leave anything blank. Choose the BEST answer.

**MCQ 1**. **(1 pt)** Function Calls & Parameters

Which call matches this function's header?

```
1 def g(s: str, n: int) -> str:
2     ...
```

*A*. `g(2, "hi")`
*B*. `g("hi")`
*C*. `g("hi", 3)`
*D*. `g()`
*E*. None of the above

**MCQ 2**. **(1 pt)** Operators & Types

What is the value of `7 // 3` in Python?

*A*. 2.333...
*B*. 2
*C*. 3
*D*. 1
*E*. None of the above

**MCQ 3**. **(1 pt)** Lists & Accumulators

Which method removes the last item from the list `lst`?

*A*. `lst.append(x)`
*B*. `lst.remove(x)`
*C*. `lst.pop()`
*D*. `lst[-1]`
*E*. None of the above

**MCQ 4**. **(1 pt)** Conditionals & Variables

What happens if you use a variable directly in a conditional before assigning it a value?

```
if y > 0:
    print("positive")
```

*A*. It prints `"positive"` by default.
*B*. It treats `y` as 0.
*C*. It skips the conditional without error.
*D*. It prints nothing.
*E*. It raises a `NameError` because `y` is not defined.

**MCQ 5**. **(1 pt)** Loops & Range

Given:

```
1 s = 0
2 for i in range(4):
3     s += i
```

What does `s` equal after the code above is run?

*A*. Sum of 0 through 3
*B*. Sum of 1 through 4
*C*. $0 + 1 + 2 + 3 + 4$
*D*. Infinite loop
*E*. None of the above

**MCQ 6**. **(1 pt)** Booleans & DeMorgan's Law

Which expression is equivalent to `not (x and y)`?

*A*. `not x and not y`
*B*. `not x or not y`
*C*. `x or y`
*D*. `not x or y`
*E*. None of the above

**MCQ 7**. **(1 pt)** Lists & Accumulators

Given:

```
1 lst = [10, 20, 30]
2 print(lst[1:])
```

What is the output?

*A*. `[10]`
*B*. `[20, 30]`
*C*. `[10, 20]`
*D*. Error
*E*. None of the above

**MCQ 8**. **(1 pt)** Function Calls & Parameters

Given:

```
1 def f(x, y=2):
2     return x * y
3 answer = f(3)
```

What is the value of `answer` after the code above is executed?

*A*. 3
*B*. 6
*C*. Error (missing parameter)
*D*. `[3,2]`
*E*. None

**MCQ 9**. **(1 pt)** Strings vs Lists & Mutation

Which statement causes an error?

*A*. `s = "cat"; s[0] = "b"`
*B*. `lst = [1, 2, 3]; lst[0] = 9`
*C*. `s = "cat"; s = "bat"`
*D*. `lst = [1, 2]; lst.append(3)`
*E*. None of the above

**MCQ 10**. **(1 pt)** Conditionals & Refactoring

Given:

```
x = 4
if x > 5:
    print("big")
elif x == 4:
    print("equal")
else:
    print("small")
```

What is printed?

*A*. `big`
*B*. `equal`
*C*. `small`
*D*. `nothing`
*E*. None of the above

**MCQ 11**. **(1 pt)** Operators & Types

Suppose `x = 7.0` and `y = 3`. What is the type of `x // y`?

*A*. `int`
*B*. `float`
*C*. `str`
*D*. `bool`
*E*. None of the above

**MCQ 12**. **(1 pt)** Booleans & DeMorgan's Law

Given the following valid fragment of code:

```
if not (n > 0 or n < 0):
```

Which conditional is equivalent to it?

*A*. `if n == 0:`
*B*. `if n != 0:`
*C*. `if not n:`
*D*. `if n > 0 and n < 0:`
*E*. None of the above

**MCQ 13**. **(1 pt)** Lists & Accumulators

Given:

```
nums = [1, 2, 3]
s = 0
for n in nums:
    s += n
print(s)
```

What is the output of this code?

*A.* 0
*B.* 3
*C.* 6
*D.* Error
*E.* None of the above

**MCQ 14**. **(1 pt)** Loops & Range

Given:

```
s = "abc"
for ch in s:
    print(ch.upper(), end="")
```

What is the output of this code?

*A.* `abc`
*B.* `ABC`
*C.* `a b c`
*D.* Error
*E.* None of the above

**MCQ 15**. **(1 pt)** Operators & Types

Which expression evaluates to `True`?

*A.* `5 % 2 == 2.5`
*B.* `10 / 3 == 3.333`
*C.* `4 // 2 == 2`
*D.* `7 / 2 == 3`
*E.* None of the above

**MCQ 16**. **(1 pt)** Conditionals & Refactoring

Which of the following is equivalent to:

```
if grade >= 90:
    letter = "A"
else:
    if grade >= 80:
        letter = "B"
    else:
        letter = "C"
```

*A*.
```
if grade >= 90:
    letter = "A"
elif grade >= 80:
    letter = "B"
else:
    letter = "C"
```

*B*.
```
if grade >= 80:
    letter = "B"
elif grade >= 90:
    letter = "A"
else:
    letter = "C"
```

*C*. Both *A*. and *B*.

*D*. Neither *A*. nor *B*.

*E*. None of the above

**MCQ 17**. **(1 pt)** Booleans & DeMorgan's Law

If `x` is an integer, which boolean expression is always `False`?

*A*. `x < 5 or x >= 5`

*B*. `x < 5 and x >= 5`

*C*. `x == 5 or x == 6`

*D*. `not(x < 0 and x > 0)`

*E*. None of the above

**MCQ 18**. **(1 pt)** Strings vs Lists & Mutation

Given:

```
lst = [1, 2, 3]
lst2 = lst
lst2[0] = 9
print(lst)
```

What is the output of this code?

*A*. `[1, 2, 3]`
*B*. `[9, 2, 3]`
*C*. `[1, 9, 3]`
*D*. Error
*E*. None of the above

**MCQ 19**. **(1 pt)** Loops & Range

Given:

```
for i in range(2, 7, 2):
    print(str(i) + " ")
```

What is the output of this code?

*A*. 2 3 4 5 6
*B*. 2 4 6
*C*. 2 4
*D*. 3 5
*E*. None of the above

**MCQ 20**. **(1 pt)** Function Calls & Parameters

Given:

```
def f(x, y=2):
    print(x * y)
answer = f(2, 4)
```

What is the value of `answer` after the code above is executed?

*A.* 4
*B.* 8
*C.* Error (missing parameter)
*D.* `[2,4]`
*E.* `None`

**MCQ 21**. **(1 pt)** Strings vs Lists & Mutation

Given:

```
s = "hello"
print(s.replace("l", "x", 1))
print(s)
```

What is the output of this code?

*A.* hexxo then hexxo
*B.* hexlo then hexlo
*C.* hexlo then hello
*D.* hello then hello
*E.* None of the above

**MCQ 22**. **(1 pt)** Something You Should Know

Which of the following is *not* one of the instructors for CSC108H5F, 20259 (Fall 2025) at UTM?

*A.* Prof. Michael Liut
*B.* Prof. Joshua Jung
*C.* Prof. Mai Ha Vu
*D.* Prof. Mohammad Mahmoud
*E.* Prof. Andrew Petersen

## Part 2: Fill in the Blanks / Short Code (10 marks).

Answer in the space provided. Write only the missing code or expression unless otherwise indicated. No partial credit.

### Q23. Boolean Refactoring (2 pts)

Rewrite the following statement without using the `or` operator:

```
1 if (x > 5 or y <= 0):
2     print("Valid")
```

```
if _____________________________________________:
    print("Valid")
```

### Q24. Conditional Simplification (2 pts)

The code below is too long:

```
1 if n % 2 == 0:
2     even = True
3 else:
4     even = False
```

Refactor the code. Rewrite it on *one line* below.
You must use a boolean expression.

```
even = ________________________________________________
```

**Q25. Using a Helper Function** (2 pts)

You are given the following helper function:

```
1 def exists_triangle(a: int, b: int, c: int) -> bool:
2     """Return True if sides <a>, <b>, <c> form a triangle."""
```

Complete the body of the function below so it returns whether the string `s` is a "triangle string." You must adhere to the header and type contract. A triangle string is formed by three consecutive groups of digits (all `1`s, then `2`s, then `3`s). Each group may contain zero or more digits. For example, the string `"1112223333"` corresponds to sides (3, 3, 4), which form a valid triangle. You may assume that every input string `s` only contains the digits `1`, `2`, and `3`.

```
def is_triangle_string(s: str) -> bool:



    return exists_triangle( ___________________________________,

                            ___________________________________,

                            ___________________________________ )
```

### Q26. Range & Accumulator (2 pts)

Complete this loop. The loop computes the product of the numbers from 1 through 4 (i.e., $1 \times 2 \times 3 \times 4$).

```
total = 1

for i in range( ______________________, ______________________ ):

    total *= i
```

### Q27. File I/O in a Loop (2 pts)

Let's assume that the file `data.txt` initially contains the following content:

```
1  init
```

Now consider this python code:

```
1  for i in range(3):
2      with open("data.txt", "w") as f:
3          f.write(f"W{i}\n")
4      with open("data.txt", "a") as f:
5          f.write(f"A{i}\n")
```

After the above code executes, what are the exact contents of `data.txt`? (i.e., write the content of `data.txt` below.)

## **Part 3**: Parsons Problems.

For each question below, you are given a set of *unordered* code blocks. Your task is to **reorder and properly indent** the blocks to form a correct solution to the described problem.

Note:

- Not all blocks may be needed; some could be *distractors*.
- Indentation matters (i.e., code must be indented correctly to run).
- To save time, each block has a **label (A, B, C, . . . )**. Write your solution as a sequence of labels with indentation shown by nesting.
- You do not need to copy/re-write all of the code again.
- If it is unsolvable based on the given code, write "UNSOLVABLE".

Sample solution (labels with indentation):

```
A
B
    C
        D
E
```

This sample solution shows that `A`, `B`, and `E` are at the same indentation level. `C` is indented under `B`, while `D` is indented further (i.e., `D` is indented once under `C` and double indented from `B`).

**Q28. Accumulator Pattern with a String** (3 pts)

**Task:** Order and indent the code blocks below to count the number of vowels in a string `s`. You may not need all of these blocks!

A. `for ch in s:`

B. `if ch in "aeiou":`

C. `count += 1`

D. `count = 0`

E. `return count`

F. `count = ""`

G. `count = count + ch`

**Q29. While Loop Simulation** (3 pts)

**Task:** Order and indent the code blocks below so they read integers from a list `nums` until a negative number is found. Return the sum of all numbers before the first negative. You may not need all of these blocks!

A. `while i < len(nums) and nums[i] >= 0:`

B. `total = 0`

C. `total += nums[i]`

D. `i = 0`

E. `return total`

F. `i += 1`

G. `while i < len(nums):`

H. `return i`

I. `print(total)`

**Q30. List Mutation vs. String Immutability** (3 pts)

**Task:** Replace all occurrences of `"a"` with `"x"` in a list of characters `lst`. You may not need all of these blocks!

A. `if lst[i] == "a":`

B. `for i in range(len(lst)):`

C. `lst[i] = "x"`

D. `return lst`

E. `s = s.replace("a", "x")`

F. `lst.append("x")`

G. `return s`

**Part 4**: Code Writing.

In this section, you will be asked to implement complete Python functions according to detailed specifications. Each function **must**:

- Use proper Python syntax, including function headers and return statements.
- NOT contain doctests (i.e., do not create doctests!).
- Produce the exact output(s) described in the examples/test-cases given.
- NOT be hardcoded (i.e., your program should calculate results based on input or logic, not use fixed values).
- be legible, written clearly, and contain all appropriate indentation(s) and style.

Your answers will be graded for correctness, clarity, and adherence to the requirements. Illegible solutions will be given a 0. Partial credit *may* be awarded for solutions that are on the right track but are incomplete or incorrect. It is always best to try a question, write pseudo code if needed, but please do not leave it blank!

Unless otherwise stated:

- Write your solution in the space provided below each question.
- Do not include additional `print` statements.
- Do not use external libraries (i.e., do not import anything).
- If you make an assumption (only when absolutely necessary), write it down as a comment.

Each Code Writing question is worth 5 marks.

**Q31. Alternating Case String** (5 pts)

**Task:** Write a function `alt_case(s: str) -> str:` that returns a new string where:

- Characters at even indices (0, 2, 4, ...) are uppercase.
- Characters at odd indices (1, 3, 5, ...) are lowercase.

The function must work for any string.

**Examples:**

```
>>> alt_case("hello")
"HeLlO"

>>> alt_case("Programming")
"PrOgRaMmInG"

>>> alt_case("a")
"A"
```

## Q32. Average Temperature and Second Hottest (10 pts)

### (a) Average Temperature [5 points]

**Task:**

Write a function `average_temperature(temps: list[float]) -> int:` that returns the average of the entries in the input list of temperatures.

- The average should be rounded to the nearest integer using Python's built-in `round` function.
- If the list is empty, the function should return `0`.
- Assume the list contains only numeric values (only numbers).

**Examples:**

```
>>> average_temperature([20.0, 21.0, 22.0])
21

>>> average_temperature([30.4, 30.6])
30

>>> average_temperature([])
0
```

**(b) Second Hottest** **[5 points]**

**Task:**

Write a function `second_hottest(temps: list[float]) -> int:` that returns the **second highest temperature** in the input list.

- You must use at least one loop.
- Do **not** use built-in functions such as `max()`, `min()`, `sorted()`, or `sort()`.
- If the list has fewer than two **distinct** temperature values, return `0`.
- The temperatures may include both positive and negative numbers.
- Recall that float("inf") represents infinity and float("-inf") represents negative infinity.

**Examples:**

```
>>> second_hottest([20.0, 21.0, 22.0])
21

>>> second_hottest([30.4, 30.6])
30

>>> second_hottest([15.0, 15.0, 15.0])
0

>>> second_hottest([])
0
```

Write your solutions in the space below (or on the next page).

**Blank Page for Rough Work**

If continuing an answer here, write "Continued on page 21." on the question page.

**Q33. Mark Boundaries** (5 pts)

**Task:** Write a function `mark_boundaries(items: list[str]) -> list[str]:` that returns a **new list** where the string `"---"` is inserted **between groups of identical consecutive items** in the input list `items`.

- Insert exactly one marker `"---"` between two groups that differ.
- Leave runs of identical items unchanged.
- Do not modify the original list `items`.
- Return an empty list `[]` if the input list is empty.

**Examples:**

```
>>> mark_boundaries(["A", "A", "B", "B", "C"])
['A', 'A', '---', 'B', 'B', '---', 'C']

>>> mark_boundaries(["X", "Y", "Z"])
['X', '---', 'Y', '---', 'Z']

>>> mark_boundaries(["Same", "Same", "Same"])
['Same', 'Same', 'Same']

>>> mark_boundaries([])
[]
```

Write your solution in the space below. You may use additional space at the back if needed.

**Blank Page for Rough Work**

If continuing an answer here, write "Continued on page 23." on the question page.

## **Bonus Question** (Optional, 0.5 pts).

For this question, we would like you to reflect on how you approached learning the course material.

- If you have used ChatGPT or another generative AI tool at any point this term, please circle this bullet point. State which tool you used and briefly (no more than 6 sentences) explain how it supported your learning compared to the textbook or lecture slides.
- If you have not used ChatGPT or another generative AI tool, please circle this bullet point. Write a short paragraph (no more than 6 sentences) explaining what course resources (e.g., lecture slides, textbook, labs, practice problems) were most helpful for you, and why.

Please write this in your own words. We are trying to understand what works best for students in this course, your answer will be used to improve the course and computing instruction worldwide. The purpose is to reflect on what has supported your learning so far, which will help us help you in this course and future courses at UTM!

## API Sheet: Selected Python Functions, List, String, & File Methods

This sheet is provided for your reference during the exam. Only the functions/methods listed here are guaranteed to be available; assume no other modules are imported.

```
__builtins__:
  enumerate(iterable, start=0)
    Return an enumerate object yielding pairs (index, value).
    enumerate is useful for obtaining an indexed list:
    (0, seq[0]), (1, seq[1]), (2, seq[2]), ...
  int(x) -> int
    Convert x to an integer, truncating toward zero if float.
  len(x) -> int
    Return the number of items in container (e.g. list or string) x.

  max(...)
        max(iterable, *[, default=obj, key=func]) -> value
        max(arg1, arg2, *args, *[, key=func]) -> value
    With a single iterable argument (e.g. list), return its biggest item.
    The default keyword-only argument specifies an object to return if the
    provided iterable is empty. With two or more arguments, return the
    largest argument.

  min(...)
        min(iterable, *[, default=obj, key=func]) -> value
        min(arg1, arg2, *args, *[, key=func]) -> value
    With a single iterable argument, return its smallest item.
    The default keyword-only argument specifies an object to return if the
    provided iterable is empty. With two or more arguments, return the
    smallest argument.

  print(*objects, sep=' ', end='\n') -> None
    Print objects separated by sep and followed by end.
  range(stop) -> range
    range(start, stop[, step]) -> range
    Return an immutable sequence of integers.
  round(number, ndigits=None) -> int | float
    Round a number to a given precision in decimal digits.
    The return value is an integer if ndigits is omitted or None.
    Otherwise, the return value has the same type as the number.
    ndigits may be negative.

list:
  L.append(x) -> None
    Add item x to the end of list L.
  L.pop([i]) -> object
    Remove and return item at index i (default last).
  L.remove(x) -> None
    Remove first occurrence of x in L; raise ValueError if not found.
  L[index] = value
    Assign value at position index in L.
  L[start:stop]
    Return a sublist from start up to (not including) stop.
```

```
str:
  x in s -> bool
    Return True if string x is a substring of s.
  str(x) -> str
    Return the string version of object x.
  S.isalpha() -> bool
    Return True if all characters in S are alphabetic.
  S.isdigit() -> bool
    Return True if all characters in S are digits.
  S.islower() -> bool
    Return True if all cased characters are lowercase.
  S.isupper() -> bool
    Return True if all cased characters are uppercase.
  S.lower() -> str
    Return a copy of S with all cased characters lowercase.
  S.upper() -> str
    Return a copy of S with all cased characters uppercase.
  S.replace(old, new[, count]) -> str
    Return a copy with all (or first count) occurrences of old replaced by new.
  S.strip([chars]) -> str
    Return a copy with leading and trailing whitespace (or chars) removed.
  S.lstrip([chars]) -> str
    Return a copy with leading whitespace (or chars) removed.
  S.rstrip([chars]) -> str
    Return a copy with trailing whitespace (or chars) removed.
  S.count(sub[, start[, end]]) -> int
    Return the number of non-overlapping occurrences of substring sub.
  S.find(sub[, start[, end]]) -> int
    Return lowest index where sub is found; -1 if not found.
  S.index(sub[, start[, end]]) -> int
    Like find(), but raise ValueError if not found.

file:
  open(file, mode='r', encoding=None) -> file object
    Open a file and return a corresponding file object.
    Common modes:
      'r'  read (default),      'w'  write (overwrite),      'a'  append,
      'b'  binary mode,         't'  text mode (default)

file object methods:
  f.read() -> str
    Return entire contents of file as a string.
  f.readline() -> str
    Return the next line from the file.
  f.readlines() -> list[str]
    Return a list of all remaining lines.
  f.write(s: str) -> int
    Write string s to file; return number of characters written.
  f.close() -> None
    Close the file.
```